

Document number	P2878R6
Date	2023-11-13
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	SG23 Safety and Security

Reference checking

Table of contents

- Reference checking
 - Changelog
 - Abstract
 - Motivational Example
 - Motivation
 - Other Proposals
 - Impact on general lifetime safety
 - Impact on Contracts
 - Technical Details
 - Recursion
 - Structs and Classes
 - Lambdas revisited
 - Impact on Pattern Matching
 - Resolution
 - Usage
 - Viral Attribution Effort
 - Summary
 - Poll Results
 - Frequently Asked Questions
 - Why not pointers?
 - Why not explicit lifetime dependence on struct(s)?
 - References

Changelog

R6

- revising comments in Impact on Pattern Matching example
- added more links to changelog for easier navigation
- added Poll Results section
- revised Viral Attribution Effort section to include the ability for the compiler to provide the attributes/metadata upon module compilation

R5

- revised resolution to the do expression/pattern matching dangling problem, explaining it iteratively
- added a 6th check for `std::thread` and `std::jthread`

R4

- revised recursive example
- revised `do return`'s `return` example
- minor formatting changes
- improved FAQ: Why not pointers?

R3

- added resolution to the do expression/pattern matching dangling problem
- added check for assigning a temporary to a named variable produces an error
- added recursive function call example to illustrate functions being analyzed in isolation
- added the Lambdas revisited example
- added a Usage section
- added a Viral Attribution Effort section

R2

- fixed `f3` example of returning a `struct`
- revised comments of 4th check example
- added resolved resolution example

Abstract

This paper proposes that we allow programmers to provide explicit lifetime dependence information to the compiler for the following reasons:

- Standardize the documentation of lifetimes of API(s) for developers
- Standardize the specification of lifetimes for proposals
- Greatly reduce the dangling of the stack for references

This paper is **NOT** about the following:

- Fixing dangling of the stack that can only be discovered at runtime
- Directly fixing dangling of the heap when not using RAI
- Fixing dangling that can occur when using pointers and pointer like types

Rather it is about making those instances of dangling references to the stack which are always bad code, detectable as errors in the language, instead of warnings.

Motivational Example

Disclaimer: I am not a RUST expert. Any RUST examples provided here is to illustrate this feature as a language feature instead of an attribute. This proposal will repeatedly refer to this as an attribute but there is no preference on which, rather it is important to gain the functionality.

What is being asked for is similar to but not exactly like Rust's feature called `explicit lifetimes`.

Example taken from `Why are explicit lifetimes needed in Rust?` ^[1]

```
fn foo<'a, 'b>(x: &'a u32, y: &'b u32) -> &'a u32 {  
    x  
}
```

Similar but better functionality has been requested in a variety of C++ proposals.

1. Bind Returned/Initialized Objects to the Lifetime of Parameters, Rev0 ^[2]

```
const unsigned int& foo(const unsigned int& x lifetimebound, const unsigned int& y
    return x;
}
```

2. indirect dangling identification [3]

```
[[parameter_dependency(dependent{"return"}, providers{"x"})]]
const unsigned int& foo(const unsigned int& x, const unsigned int& y) {
    return x;
}
```

3. Towards memory safety in C++ [4]

```
[[dependson(x)]] const unsigned int& foo(const unsigned int& x, const unsigned int
    return x;
}
```

Rust also allows providing explicit lifetimes on struct(s) but that is **NOT** being asked for in this proposal.

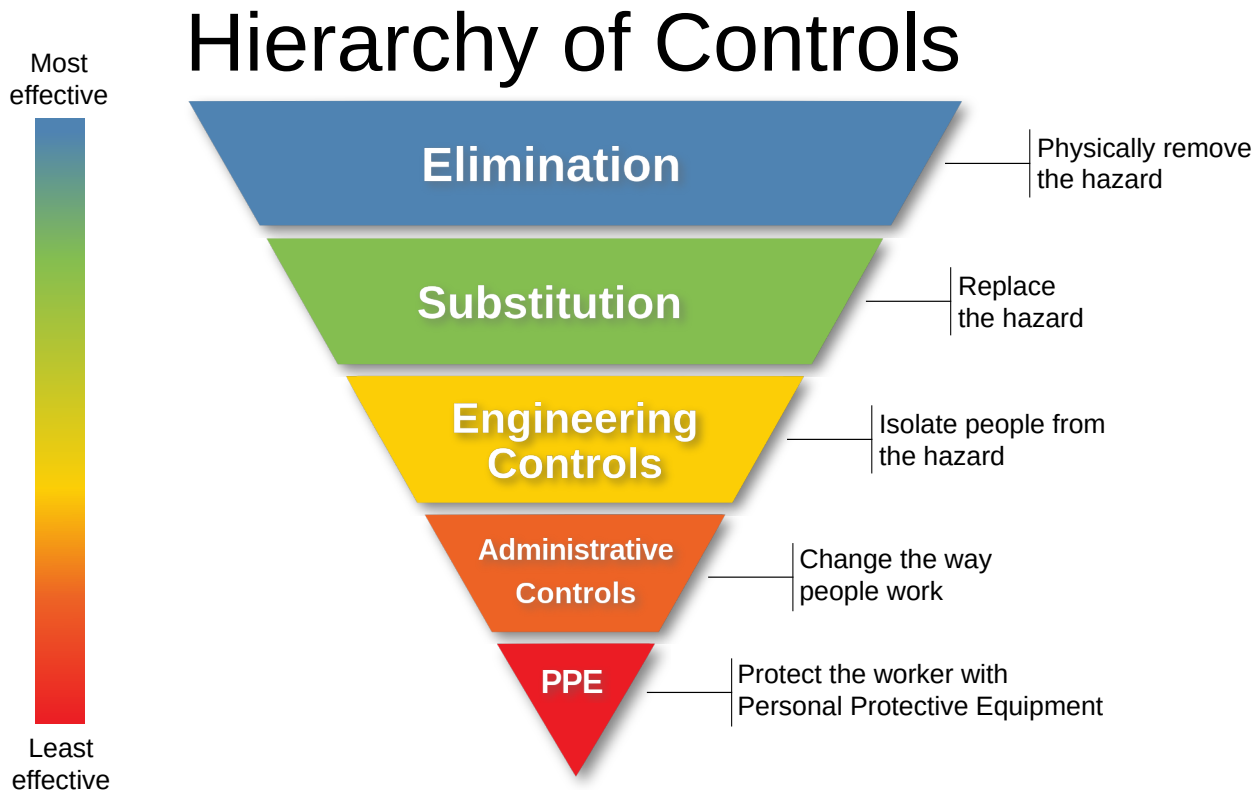
Example taken from [Why are explicit lifetimes needed in Rust?](#) [1:1]

```
struct Foo<'a> {
    x: &'a i32,
}

fn main() {
    let f : Foo;
    {
        let n = 5; // variable that is invalid outside this block
        let y = &n;
        f = Foo { x: y };
    };
    println!("{}", f.x);
}
```

Motivation

Having these checks in the language is highly desirable because it is highly effective. Other proposals are advocating for a separate tool whether that be a static analysis tool or a runtime tool. Those solutions, while needed, are less effective because they are at best only PPE, personal protective equipment that only works if you have it installed, turned on, configured, used and acted upon. This proposal, while limited, is highly effective because it eliminates these instances of safety issues as illustrated on the *[exponential]* safety scale [5].



This proposal is also in line with what that C++ community believes and teaches.

“In.force: Enforcement” ^[6]

...

*“This adds up to quite a few dilemmas. We try to resolve those using tools. Each rule has an **Enforcement** section listing ideas for enforcement. **Enforcement might be done by code review, by static analysis, by compiler, or by run-time checks. Wherever possible, we prefer ‘mechanical’ checking (humans are slow, inaccurate, and bore easily) and static checking. Run-time checks are suggested only rarely where no alternative exists; we do not want to introduce ‘distributed bloat’.**”* ^[6:1]

...

P.5: Prefer compile-time checking to run-time checking ^[7]

C++ Core Guidelines ^[6:2]

Bjarne Stroustrup, Herb Sutter

Other Proposals

Before going into the technical details, it would be good to consider this in light of other safety related proposals.

Impact on general lifetime safety

This proposal does not conflict with **Lifetime safety: Preventing common dangling** ^[8]. To the contrary, it is complimentary.

***“1.1.4 Function calls”** “Finally, since every function is analyzed in isolation, we have to have some way of reasoning about function calls when a function call returns a Pointer. If the user doesn’t annotate otherwise, by default we assume that a function returns values that are derived from its arguments.”*

Lifetime safety: Preventing common dangling ^[8:1]

Herb Sutter

Just like the Lifetime safety: Preventing common dangling ^[8:2] paper, this proposal analyze each function in isolation. While that proposal is focused on when programmers do not annotate/document their code, this proposal is focused on when programmers do document their code. That proposal “*assume that a function returns values that are derived from its arguments*” and because of this assumption has to produce warnings. This proposal makes no assumption at all and consequently can produce errors from the library authors documented intentions. This is both complimentary, independent and both proposals are desired by the C++ community. Since both related features are independent than there is no reason why the compile checks proposal couldn't be added before the runtime checks proposal.

Impact on Contracts

This proposal also does not conflict with contracts. Matter of fact, this proposal could be described in terms of contracts, just applied to existing language features. Since, none of the current contract proposals allow applying contracts to existing and future non functional language features such as `return` and `do return` ^[9], this proposal is complimentary.

“7.1 LIFETIME” “*These sources of undefined behavior pertain to accessing an object outside its lifetime or validity of a pointer. By their very nature, they are not directly syntactic. The approach suggested in this proposal is to prohibit the use of certain syntactic constructs which might – under the wrong circumstances – lead to undefined behavior. Those restrictions are syntactic, so clearly will prohibit cases that someone might find useful.*” ^[10]

Contracts for C++: Prioritizing Safety ^[10:1]

Gabriel Dos Reis

The Contracts for C++: Prioritizing Safety ^[10:2] proposal envisions a predicate called `object_address` that could be applied via contracts to functions. In contract like terms, this proposal would be advocating for `is_global_object_address` , `is_local_object_address` , `is_temporary_object_address` , `is_not_global_local_temporary_object_address` and then applying these predicates to `return` , reference dereference and the points of reference use. Since all of this is compile time information and the places where applied would always produce errors than there is no need for the programmer to add such checks anywhere because the compiler can do it automatically. Further, as this proposal only serve to identify code that is definitely bad, then this proposal does not “*prohibit cases that someone might find useful*”.

Technical Details

In order to make this proposal work, 2 bits of information is needed per reference at compile time. The 2 bits represents an enumeration of 4 possible lifetime values.

1. Is global
2. Is local
3. Is temporary
4. Is all other i.e. not global, local or temporary? i.e. unknown, `nullptr` and dynamic

This lifetime enumeration gets associated with each reference at the point of construction since references have to be initialized, can't be `nullptr` and can't be rebound.

```
const int GLOBAL = 42;

void f(int* ip, int& ir/* unknown lifetime */)
{
    int local = 42;
    int& r1 = *ip;// unknown lifetime
    int& r2 = ir;// unknown lifetime
    int& r3 = GLOBAL;// global lifetime
    int& r4 = local;// local lifetime
}
```

The next step is copying a reference copies its lifetime metadata.

```
const int GLOBAL = 42;

void f(int* ip, int& ir/* unknown lifetime */)
{
    int local = 42;
    int& r1 = *ip;// unknown lifetime
    int& r2 = ir;// unknown lifetime
    int& r3 = GLOBAL;// global lifetime
    int& r4 = local;// local lifetime
    int& r5 = r1;// unknown lifetime
    int& r6 = r2;// unknown lifetime
    int& r7 = r3;// global lifetime
    int& r8 = r4;// local lifetime
}
```

1st check: Returning a reference to a local produces an error.


```

const int GLOBAL = 42;

int& f(int* ip, int& ir/* unknown lifetime */)
{
    int local = 42;
    int& r1 = *ip;// unknown lifetime
    int& r2 = ir;// unknown lifetime
    int& r3 = GLOBAL;// global lifetime
    int& r4 = local;// local lifetime
    int& r5 = r1;// unknown lifetime
    int& r6 = r2;// unknown lifetime
    int& r7 = r3;// global lifetime
    int& r8 = r4;// local lifetime
    return r8;// error: dangling a local
}

```

This error doesn't give programmers much as C++ addressed this in C++23 with `simpler implicit move` ^[11]. However, since `simpler implicit move` ^[11:1] was framed in terms of value categories than any error message would also be in terms of value categories. This proposal advises such an error would be expressed in terms of dangling which is more human readable for programmers. It also allows infinitely levels of indirection.

Things get really interesting when programmers are allowed to provide explicit lifetime dependence information to the compiler. This feature, `explicit lifetime dependence`, allows a reference to be tied to the lifetimes of multiple other references. In these cases, the lifetime is the most constrained as in `temporary` is more constrained than `local` which is more constrained than `global`.

`global > local > temporary`

This is also time to add three more checks.

2nd check: Returning a reference to a `temporary` produces an error.

3rd check: Using a reference to a `temporary` after it has been assigned, i.e. on another line of code which is not the full-expression, produces an error.

4th check: Assigning a `temporary` to a named variable produces an error.

```
const int GLOBAL = 42;
```

```
[[dependson(left, right)]]
```

```
const int& f1(const int& left/* unknown lifetime */, const int& right/* unknown li
{
    if(randomBool())
    {
        return left;
    }
    else
    {
        return right;
    }
}
```

```
int& f2()
```

```
{
    int local = 42;
    const int& r1 /*local*/ = f1(local, local);
    const int& r2 /*global*/ = f1(GLOBAL, GLOBAL);
    const int& r3 /*temporary*/ = f1(42, 42);// error: can't assign temporary to a
    const int& r4 /*local*/ = f1(local, GLOBAL);
    const int& r5 /*temporary*/ = f1(local, 42);// error: can't assign temporary t
    const int& r6 /*temporary*/ = f1(GLOBAL, 42);// error: can't assign temporary
    if(randomBool())
    {
        return r1;// error: can't return local
    }
    if(randomBool())
    {
        return r2;// OK its a reference to a global
    }
    if(randomBool())
    {
        return r3;// error: can't return temporary
    }
    if(randomBool())
    {
        return r4;// error: can't return local
    }
    if(randomBool())
    {
        return r5;// error: can't return temporary
    }
    if(randomBool())
    {
```

```

        return r6; // error: can't return temporary
    }
    int x1 = r3 + 43; // error: can't use reference to a temporary
    int x2 = r5 + 44; // error: can't use reference to a temporary
    int x3 = r6 + 45; // error: can't use reference to a temporary
    return f1(f1(GLOBAL, 4), f1(local, 2)); // error: can't return temporary
}

```

Besides fixing indirect dangling of a local, this also fixes indirect dangling of temporaries which causes immediate dangling.

6.7.7 Temporary objects [class.temporary]

...

⁴ When an implementation introduces a temporary object of a class that has a non-trivial constructor (11.4.5.2, 11.4.5.3), it shall ensure that a constructor is called for the temporary object. Similarly, the destructor shall be called for a temporary with a non-trivial destructor (11.4.7). **Temporary objects are destroyed as the last step in evaluating the full-expression** (6.9.1) that (lexically) contains the point where they were created. This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full-expression, not with any specific subexpression.

...

(6.12) - A temporary bound to a reference in a *new-initializer* (7.6.2.8) persists until the completion of the **full-expression** containing the *new-initializer*.

[*Note 7: This might introduce a dangling reference. - end note*]

Working Draft, Standard for Programming Language C++ ^[12]

Recursion

Before going further, let's consider a recursive example in order to illustrate why each function is considered in isolation.

```
const int GLOBAL = 42;
```

```
[[dependson(input)]]
```

```
const int& recursive(const int& input/* unknown lifetime */)
{
    if(randomBool())
    {
        return GLOBAL;// globals always fine
    }
    int local = randomInt();
    if(randomBool())
    {
        return local;// error: can't return local
    }
    if(randomBool())
    {
        return 42;// error: can't return temporary
    }
    if(randomBool())
    {
        return input;// handled by attribute lower on call stack
    }
    return recursive(input);// unknown
    // doesn't matter
    // local's and temporaries lower on the stack are safe
    // dynamic would have to be cleaned up after the fact anyway
    // and if dynamic was deleted prior than that would be a problem
    // prior to this call; in all cases caller's responsibility
}
```

```
int& f()
```

```
{
    int local = 42;
    const int& r1 /*global*/ = recursive(GLOBAL);
    const int& r2 /*local*/ = recursive(local);
    const int& r3 /*temporary*/ = recursive(42);// error: can't assign temporary t
    if(randomBool())
    {
        return r1;// OK, its a reference to a global
    }
    if(randomBool())
    {
        return r2;// error: can't return local
    }
    if(randomBool())
    {

```

```
        return r3;// error: can't return temporary
    }
    int x1 = r3 + 43;// error: can't use reference to a temporary
    return recursive(42);// error: can't return temporary
}
```

If we fixed nothing else identified in this proposal, this would be a welcome reprieve. However, much more can be done simply. Let's say, instead of adding this lifetime metadata to each reference, we add it to each instance. For references, lifetime metadata would still say that the reference refers to an instance with a particular lifetime but for non reference and non pointer instances, lifetime metadata would indicate that the instance is dependent upon another instance. Let's see what type of dangling this would mitigate.

Structs and Classes

Before adding this metadata to instances in general, consider a struct that contains references. This shows that the composition and decomposition of references can be handled by the compiler provided the reference is accessible such as `public`.

```

struct S { int& first; const int& second; };

const int GLOBAL = 42;

[[dependson(left, right)]]
const int& f1(const int& left/* unknown lifetime */, const int& right/* unknown li
{
    if(randomBool())
    {
        return left;
    }
    else
    {
        return right;
    }
}

int& f2()
{
    int local = 42;
    S s1{GLOBAL, local};
    S s2{local, f1(GLOBAL, 24)}; // error: can't assign temporary to a named instan
    const int& r1 /*global*/ = s1.first;
    const int& r2 /*local*/ = s1.second;
    const int& r3 /*local*/ = s2.first;
    const int& r4 /*temporary*/ = s2.second; // error: can't assign temporary to a
    if(randomBool())
    {
        return r1; // OK its a reference to a global
    }
    if(randomBool())
    {
        return r2; // error: can't return local
    }
    if(randomBool())
    {
        return r3; // error: can't return local
    }
    if(randomBool())
    {
        return r4; // error: can't return temporary
    }
    int x = r4 + 43; // error: can't use reference to a temporary
    return 42; // error: can't return temporary
}

```

```

S f3()
{
    int local = 42;
    S s1{GLOBAL, local};
    S s2{local, f1(GLOBAL, 24)}; // error: can't assign temporary to a named instan
    if(randomBool())
    {
        return s1; // error: still returning a local
    }
    return s2; // error: still returning a local
    // error: still returning a temporary
}

// prevents returning lambda that captures or returns a local or temporary
auto lambda()
{
    int local = 42;
    const int& ref_temporary = f1(GLOBAL, 24); // error: can't assign temporary to
    return [&local, &ref_temporary]() -> const int&
    {
        if(randomBool())
        {
            return local; // error: can't return local
        }
        return ref_temporary; // error: can't return temporary
    }; // error: still returning a local
    // error: still returning a temporary
}

// prevents returning coroutine that captures or returns a local or temporary
auto coroutine()
{
    int local = 42;
    const int& ref_temporary = f1(GLOBAL, 24); // error: can't assign temporary to
    return [&local, &ref_temporary]() -> generator<const int&>
    {
        if(randomBool())
        {
            co_return local; // error: can't return local
        }
        co_return ref_temporary; // error: can't return temporary
    }; // error: still returning a local
    // error: still returning a temporary
}

```

Is there any way we can mitigate dangling when the reference has been hidden by abstractions such as for reference like classes via `protected`, `private` access and `public` accessors. If lifetime metadata are also applied to instances in general, then constructors, conversion operators and factory functions could be annotated to say a returned reference like type is dependent upon another type. Consider the `std::string_view` and `std::string`.

GIVEN

```
constexpr std::string::operator [[dependson(this)]] std::basic_string_view<CharT,
```

```
std::string_view sv /* dependent upon temporary */ = "hello world"s; // error: can't  
sv.size(); // error: can't use a temporary outside of the line it was created on
```

This would also work with other reference like types such as `std::span` and `function_ref` [13]. Just as this proposal does not address pointers because they are more run time than compile time, this proposal would not address `std::reference_wrapper` since it is rebindable. Nor would this proposal address, `std::shared_ptr` and `std::weak_ptr` which like pointers are nullable and rebindable, even though they already have runtime safeties built in.

Even though this proposal does not address pointers and pointer like types, it is still useful to non owning versions of these constructs when they are `const` constructed because they would need to be non null initialized to be usable and wouldn't be rebindable because they were `const`. For instance, `std::reference_wrapper` is rebindable like a pointer so it tends to be more runtime than compile time. However, a `const std::reference_wrapper` can't be rebound so it would make sense if a programmer bound its lifetime to another instance.


```
[[dependson(left, right)]]
const std::reference_wrapper<const int> f(const int& left/* unknown lifetime */, c
{
    if(randomBool())
    {
        return std::cref(left);
    }
    else
    {
        return std::cref(right);
    }
}
```



This is also time for our next check.

5th check: You can't use a temporary in a `new` expression if the type being instantiated will become dependent upon that temporary.

6.7.7 Temporary objects [class.temporary]

...

4 ... (11.4.7). Temporary objects are destroyed as the last step in evaluating the full-expression (6.9.1) ...

...

(6.12) - A temporary bound to a reference in a *new-initializer* (7.6.2.8) persists until the completion of the full-expression containing the *new-initializer*.

[*Note 7: This might introduce a dangling reference. - end note*]

[*Example 5:*

```
struct S { int mi; const std::pair<int,int>& mp; };  
S a { 1, {2,3} };  
S* p = new S{ 1, {2,3} }; // creates dangling reference
```

– *end example*]

Working Draft, Standard for Programming Language C++ ^[12:1]

In the previous example an instance of type `S` became dependent upon the temporary, `{2, 3}`, because it retained a reference to the temporary. This is detectable because the member `mp` is publicly available since the type `S` is a struct.

For a class where members `mi` and `mp` are abstracted away, the dependence can be expressed in its constructor.

```
class S
{
public:
    [[parameter_dependency(dependent{"this"}, providers{"mp"})]]
    S(int mi, const std::pair<int,int>& mp);
private:
    int mi;
    const std::pair<int,int>& mp;
};

S a { 1, {2,3} }; // error: can't assign temporary to a named instance
some_other_function(a); // error: can't use `a` as it is dependent upon a temporary
S* p = new S{ 1, {2,3} }; // error: instance dependent upon temporary
```

Lambdas revisited

Lambdas can be expressed in terms of the previous examples in two different ways depending upon whether the lambdas are capturing or not.

```

struct S
{
    int& first;
    const int& second;
    [[dependson(input)]]
    const int& f(const int& input/* unknown lifetime */);
};

[[dependson(input)]]
const int& f(const int& input/* unknown lifetime */);

// a non capturing lambda is no different than a function
// and in this case this `lambda` is essentially the same as `f`
auto non_capturing_lambda()
{
    int local = 42;
    auto lambda = [](const int& input) -> [[dependson(input)]] const int&
    {
        return input;// OK, will be handled by caller
    };
    if(randomBool())
    {
        return lambda(local);// error: can't return local
    }
    return lambda(24);// error: can't return temporary
}

// a capturing lambda is no different than a functor
// and in this case this `lambda` is essentially the same as an instance of `S`
auto capturing_lambda()
{
    int local = 42;
    const int& ref_temporary = f1(GLOBAL, 24);// error: can't assign temporary to
    auto lambda = [&local, &ref_temporary](const int& input) -> [[dependson(input)]]
    {
        if(randomBool())
        {
            return local;// error: can't return local
        }
        if(randomBool())
        {
            return ref_temporary;// error: can't return temporary
        }
        return input;// OK, will be handled by caller
    };
    if(randomBool())

```

```

{
    return lambda(local); // error: can't return local
}
return lambda(24); // error: can't return temporary
}

```

This set of examples illustrates that lambdas are no different than the previous function and class examples other than that they are anonymous and the compiler has greater responsibility in handling dangling correctly.

Impact on Pattern Matching

This feature can be enhanced further to work with potential future C++ language features such as pattern matching.

WARNING: If pattern patching can allow propagating references from inner scopes to containing scopes then there will be a new category of dangling added to C++ and consequently a weakening of the safety of references.

This paper references the `do expressions` ^[9:1] paper for the relevant portion of pattern matching because it is more explicit. If the C++ standard goes with an implicit syntax it can still be an issue if it allow propagating references from inner scopes to containing scopes.

The simplest example is as follows.

```
int x = do { do return 42; };
```

The simplest example we have to be on guard against is as follows.

```
int& x = do
{
    do return 42; // dangling likely caught by `Simpler implicit move`
};

```

This paper can similarly handle the indirect cases as was performed for returns.

```

const int& x = do
{
    int local = 42;
    const int& r1 /*local*/ = f1(local, local);
    const int& r2 /*global*/ = f1(GLOBAL, GLOBAL);
    const int& r3 /*temporary*/ = f1(42, 42);// error: can't assign temporary to a
    const int& r4 /*local*/ = f1(local, GLOBAL);
    const int& r5 /*temporary*/ = f1(local, 42);// error: can't assign temporary t
    const int& r6 /*temporary*/ = f1(GLOBAL, 42);// error: can't assign temporary
    if(randomBool())
    {
        return r1;// error: can't `return` local
    }
    if(randomBool())
    {
        return r2;// OK its a reference to a global
    }
    if(randomBool())
    {
        return r3;// error: can't `return` or `do return` a temporary
    }
    if(randomBool())
    {
        return r4;// error: can't `return` local
    }
    if(randomBool())
    {
        return r5;// error: can't `return` or `do return` a temporary
    }
    if(randomBool())
    {
        return r6;// error: can't `return` or `do return` a temporary
    }
    int x1 = r3 + 43;// error: can't use reference to a temporary
    int x2 = r5 + 44;// error: can't use reference to a temporary
    int x3 = r6 + 45;// error: can't use reference to a temporary
    return f1(f1(GLOBAL, 4), f1(local, 2));// error: can't `return` or `do return`
};

```

Do expressions i.e. pattern matching expressions are not completely self contained like functions. They can directly use locals from containing scopes. Do returning these locals **COULD** be allowed, while do returning locals in the same scope of the do expression

should **DEFINITELY** be disallowed.

```
const int& f()
{
    int local1 = 42;
    const int& rint1 = do
    {
        int local2 = 42;
        const int& rint2 = do
        {
            int local3 = 42;
            if(randomBool())
            {
                do return local1;// OK, definitively safe
            }
            if(randomBool())
            {
                do return local2;// OK for now, problem later
            }
            do return local3;// error: can't do return local
        };
        if(randomBool())
        {
            do return local1;// OK, definitively safe
        }
        if(randomBool())
        {
            do return local2;// error: can't do return local
        }
        do return rint2;// ?error or runtime unknown? could be local1 or local2
    };
}
```

The problem on `rint2` is that what should its lifetime response be; an error or unknown? The fact is even that lifetime flag can remain a local and the dangling can be identified by adding an integer to the two bits of compile time metadata.

1. First of all, each scope is given a `level` identifier .
2. Multiple scopes will share the same number if they are on the same level.
3. Starting at 1, each number is applied to each inner scope in a `breadth first search` i.e. `level order` manner.

4. Next each local also receive a `level identifier` equal to the `level identifier` of the scope they were created in. Similarly, references gets a `level identifier` equal to the `level identifier` of the instance that they reference.
5. While this metadata is meant only for locals, it may make it easier in one's implementations to assign 0 for globals and max int for temporaries.
6. A reference returned from a `do` is set to the maximum value of all the `level identifier`'s of all the references and variables `do` returned. **NOTE:** Inner `do` (s) must be evaluated before containing `do` (s).
7. Errors result from `do` returns if the variable or reference has a `level identifier` greater than or equal the the `do` return (s) `do`'s `level identifier`.
8. Errors also result from `do` (s) if the aggregated `level identifier` is greater than or equal to the `do`'s `level identifier`.

Let's iteratively apply this to the previous example.

1. First of all, each scope is given a `level identifier`.
2. Multiple scopes will share the same number if they are on the same level.
3. Starting at 1, each number is applied to each inner scope in a `breadth first search` i.e. `level order` manner.


```

const int& f()
{
  // #1, #3: scope level 1
  int local1 = 42;
  const int& rint1 = do
  {
    // #1, #3: scope level 2
    int local2 = 42;
    const int& rint2 = do
    {
      // #1, #2, #3: scope level 3
      int local3 = 42;
      if(randomBool())
      {
        // #1, #2, #3: scope level 4
        do return local1;
      }
      if(randomBool())
      {
        // #1, #2, #3: scope level 4
        do return local2;
      }
      do return local3;
    };
    if(randomBool())
    {
      // #1, #2, #3: scope level 3
      do return local1;
    }
    if(randomBool())
    {
      // #1, #2, #3: scope level 3
      do return local2;
    }
    do return rint2;
  };
}

```

4. Next each local also receive a `level identifier` equal to the `level identifier` of the scope they were created in. Similarly, references gets a `level identifier` equal to the `level identifier` of the instance that they reference.
5. While this metadata is meant only for locals, it may make it easier in one's implementations to assign 0 for globals and max int for temporaries.

```

const int& f()
{
  // #1, #3: scope level 1
  int local1 = 42; // #4: local with scope level 1
  const int& rint1 = do
  {
    // #1, #3: scope level 2
    int local2 = 42; // #4: local with scope level 2
    const int& rint2 = do
    {
      // #1, #2, #3: scope level 3
      int local3 = 42; // #4: local with scope level 3
      if(randomBool())
      {
        // #1, #2, #3: scope level 4
        do return local1;
      }
      if(randomBool())
      {
        // #1, #2, #3: scope level 4
        do return local2;
      }
      do return local3;
    };
    if(randomBool())
    {
      // #1, #2, #3: scope level 3
      do return local1;
    }
    if(randomBool())
    {
      // #1, #2, #3: scope level 3
      do return local2;
    }
    do return rint2;
  };
}

```

6. A reference returned from a `do` is set to the maximum value of all the `level` identifier 's of all the references and variables `do` return ed. **NOTE:** Inner `do` (s) must be evaluated before containing `do` (s).

```

const int& f()
{
  // #1, #3: scope level 1
  int local1 = 42; // #4: local with scope level 1
  const int& rint1 = do // #6: reference with scope level 3
  // reference scope level 3 = max(1, 2, 3) from the do return(s)
  {
    // #1, #3: scope level 2
    int local2 = 42; // #4: local with scope level 2
    const int& rint2 = do // #6: reference with scope level 3
    // reference scope level 3 = max(1, 2, 3) from the do return(s)
    {
      // #1, #2, #3: scope level 3
      int local3 = 42; // #4: local with scope level 3
      if(randomBool())
      {
        // #1, #2, #3: scope level 4
        do return local1;
      }
      if(randomBool())
      {
        // #1, #2, #3: scope level 4
        do return local2;
      }
      do return local3;
    };
    if(randomBool())
    {
      // #1, #2, #3: scope level 3
      do return local1;
    }
    if(randomBool())
    {
      // #1, #2, #3: scope level 3
      do return local2;
    }
    do return rint2;
  };
}

```

7. Errors result from `do return s` if the variable or reference has a level identifier greater than or equal the the `do return (s)` `do`'s level identifier .
8. Errors also result from `do (s)` if the aggregated level identifier is greater than or equal to the `do`'s level identifier .

```

const int& f()
{
    // #1, #3: scope level 1
    int local1 = 42; // #4: local with scope level 1
    const int& rint1 = do // #6: reference with scope level 3
    // reference scope level 3 = max(1, 2, 3) from the do return(s)
    // #8: error at the do: can't return reference to one or more locals
    // rint1's scope level 3 >= this do's scope level 2
    {
        // #1, #3: scope level 2
        int local2 = 42; // #4: local with scope level 2
        const int& rint2 = do // #6: reference with scope level 3
        // reference scope level 3 = max(1, 2, 3) from the do return(s)
        // #8: error at the do: can't return reference to one or more locals
        // rint2's scope level 3 == this do's scope level 3
        {
            // #1, #2, #3: scope level 3
            int local3 = 42; // #4: local with scope level 3
            if(randomBool())
            {
                // #1, #2, #3: scope level 4
                do return local1; // #7: OK, definitively safe
                // local1's scope level 1 < this do's scope level 3
            }
            if(randomBool())
            {
                // #1, #2, #3: scope level 4
                do return local2; // #7: OK for now, problem later
                // local2's scope level 2 < this do's scope level 3
            }
            do return local3; // #7: error: can't do return local
            // local3's scope level 3 == this do's scope level 3
        };
        if(randomBool())
        {
            // #1, #2, #3: scope level 3
            do return local1; // #7: OK, definitively safe
            // local1's scope level 1 < this do's scope level 2
        }
        if(randomBool())
        {
            // #1, #2, #3: scope level 3
            do return local2; // #7: error: can't do return local
            // local2's scope level 2 == this do's scope level 2
        }
        do return rint2; // #7: error: can't do return local
        // rint2's scope level 3 >= this do's scope level 2
    };
}

```

This is also time to mention our final check.

6th check: You can't pass a temporary to `std::thread` and `std::jthread`.

While the usefulness of this proposal for multithreading in general is limited because each function is considered in isolation, the fact we get any benefit here is some icing on the cake.

```
#include <iostream>
#include <thread>

using namespace std;

[[dependson(input)]]
const int& temporary_factory(const int& input/* unknown lifetime */)
{
    return input;
}

std::thread run() {
    const int& r1 /*temporary*/ = temporary_factory(42);
    std::thread t1([&](const int& r2) {
        std::cout << "r1 and r2 is a temporary " << r1 << r2 << std::endl;
    }, r1); // error lambda captures a temporary
    // error lambda is given a temporary upon call
    std::jthread t2([&](const int& r2) {
        std::cout << "r1 and r2 is a temporary " << r1 << r2 << std::endl;
    }, r1); // error lambda captures a temporary
    // error lambda is given a temporary upon call
    return t1;
}

int main() {
    std::thread t = run();
    t.join();
}
```

In both threads of the above example, errors are produced because the lambda gets flagged as temporary because they capture a temporary. Further, the parameter passed to temporary when it is called is also a temporary.

Consequently, a lot of definitively bad dangling has been identified and made more transparent.

So, how does this proposal stack up to the design group's opinion on safety for C++.

- *Do not radically break backwards compatibility – compatibility is a key feature and strength of C++ compared to more modern and more fashionable languages.* ^[14]
 - **This proposal does not break any correct code. It only produces errors for code that definitely dangle.**
- Do not deliver safety at the cost of an inability to express the abstractions that are currently at the core of C++ strengths. ^[14:1]
 - **This proposal does not compromise nor remove any of C++ features so abstractions are still there.**
- Do not leave us with a “safe” subset of C that eliminates C++’s productivity advantages. ^[14:2]
 - **This proposal works with all existing code. It is purely optin.**
- Do not deliver a purely run-time model that imposes overheads that eliminate C++’s strengths in the area of performance. ^[14:3]
 - **This proposal is completely compile time.**
- Do not imply that there is exactly one form of “safety” that must be adopted by all. ^[14:4]
 - **This proposal does not conflict with existing safety designs. While most others are runtime, this proposal is purely compile time. As such they are complimentary.**
- Do not promise to deliver complete guaranteed type-and-resource safety for all uses. ^[14:5]
 - **This proposal only address dangling of the stack via references which can be easily achieved if library authors provide the compiler a little more information that is needed by library users anyway.**
- Do offer paths to gradual and partial adoption – opening paths to improving the billions of lines of existing C++ code. ^[14:6]
 - **This proposal is opt in on a per function basis.**
- Do not imply a freeze on further development of C++ in other directions. ^[14:7]
 - **Runtime checks can be performed concurrently.**
- Do not imply that equivalent-looking code written in different environments will have different semantics (exception: some environments may give some code “undefined behavior” while others give it (a single!) defined behavior). ^[14:8]
 - **Bad code is bad code regardless of the environment. This proposal makes such bad code more transparent.**

DG OPINION ON SAFETY FOR ISO C++ ^[14:9]

H. Hinnant, R. Orr, B. Stroustrup, D. Vandevoorde, M. Wong

Resolution

Now that these types of dangling can be detected, there are some tools that could be provided to developers to make it easier to fix these detected instances of dangling which are **NOT** a part of this proposal. Let's go back to our first example.


```
const int GLOBAL = 42;
```

```
[[dependson(left, right)]]
```

```
const int& f1(const int& left/* unknown lifetime */, const int& right/* unknown li
{
    if(randomBool())
    {
        return left;
    }
    else
    {
        return right;
    }
}
```

```
const int& f2()
```

```
{
    int local = 42;
    const int& r1 /*local*/ = f1(local, local);
    const int& r2 /*global*/ = f1(GLOBAL, GLOBAL);
    const int& r3 /*temporary*/ = f1(42, 42);// error: can't assign temporary to a
    const int& r4 /*local*/ = f1(local, GLOBAL);
    const int& r5 /*temporary*/ = f1(local, 42);// error: can't assign temporary t
    const int& r6 /*temporary*/ = f1(GLOBAL, 42);// error: can't assign temporary
    if(randomBool())
    {
        return r1;// error: can't return local
    }
    if(randomBool())
    {
        return r2;// OK its a reference to a global
    }
    if(randomBool())
    {
        return r3;// error: can't return temporary
    }
    if(randomBool())
    {
        return r4;// error: can't return local
    }
    if(randomBool())
    {
        return r5;// error: can't return temporary
    }
    if(randomBool())
    {
```

```

        return r6; // error: can't return temporary
    }
    int x1 = r3 + 43; // error: can't use reference to a temporary
    int x2 = r5 + 44; // error: can't use reference to a temporary
    int x3 = r6 + 45; // error: can't use reference to a temporary
    return f1(f1(GLOBAL, 4), f1(local, 2)); // error: can't return temporary
}

```

Since locals and temporaries should not be returned from functions, most functions that possess this type of dangling may be in need of some refactoring, perhaps using movable value types. For dangling that occurs in the body of a function, locals need to be moved up in scope and temporaries need to be changed into locals and then perhaps moved up in scope. This results in more lines of code, superfluous naming and excessive refactoring. If the fixed temporary is only ever used in a constant fashion and if it is a literal type and constant initialized then it would likely be manually turned into a global and moved far from the point of use. All of this could be made easier upon programmers with the following features.

1. Temporaries that are initially constant referenced, where the type is a literal type and the instance could be constant initialized, then the compiler would automatically promote these to having static storage duration ^[15] just like a string and hopefully in the future like `std::initializer_list` ^[16].
2. C23 introduced storage-class specifiers for compound literals. ^[17] If C++ followed suit, then we could be able to apply `static` and `constexpr` to our temporaries. Since these two would frequently be used together it could be shortened to `constant` or `constinit`. C++ could go even farther by introducing a new specifier perhaps called `var` for variable scope that would turn the temporary into a anonymously named variable with the same life of the left most instance in the full expression. ^[18]

Let's see how these resolutions simply fixes some of the dangling in the previous example. In the following example, the keyword `constinit` is used to represent explicit constant initialization while the keyword `variable` is used to represent variable scope i.e. explicit lifetime extension. All of the `constinit` keywords in the example could be removed if there was implicit constant initialization.

```
const int GLOBAL = 42;
```

```
[[dependson(left, right)]]
```

```
const int& f1(const int& left/* unknown lifetime */, const int& right/* unknown li
{
    if(randomBool())
    {
        return left;
    }
    else
    {
        return right;
    }
}
```

```
const int& f2()
```

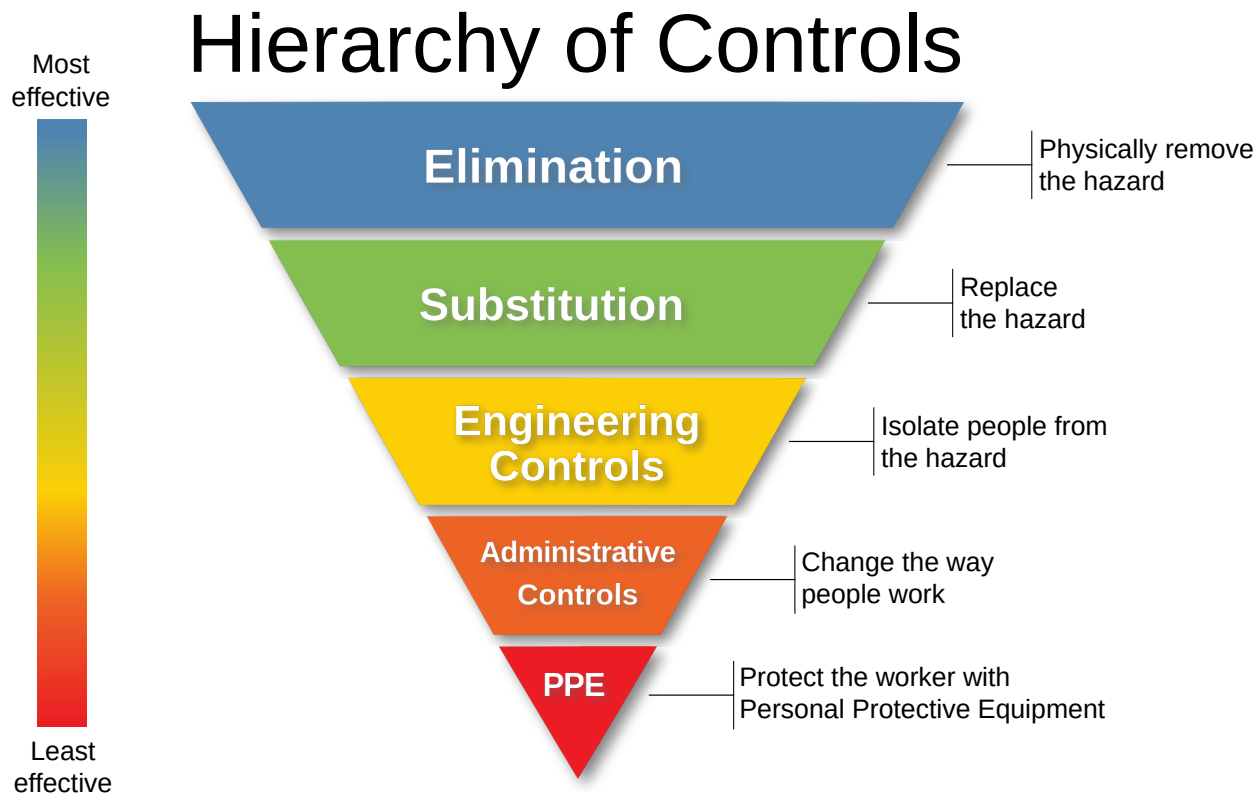
```
{
    int local = 42;
    const int& r1 /*local*/ = f1(local, local);
    const int& r2 /*global*/ = f1(GLOBAL, GLOBAL);
    const int& r3 /*local*/ = f1(constinit 42, variable 42);// was temporary
    const int& r4 /*local*/ = f1(local, GLOBAL);
    const int& r5 /*local*/ = f1(local, constinit 42);// was temporary
    const int& r6 /*local*/ = f1(GLOBAL, variable 42);// was temporary
    if(randomBool())
    {
        return r1;// error: can't return local
    }
    if(randomBool())
    {
        return r2;// OK its a reference to a global
    }
    if(randomBool())
    {
        return r3;// error: can't return local// NOTE: was temporary
    }
    if(randomBool())
    {
        return r4;// error: can't return local
    }
    if(randomBool())
    {
        return r5;// error: can't return local// NOTE: was temporary
    }
    if(randomBool())
    {
```

```
        return r6;// error: can't return local// NOTE: was temporary
    }
    int x1 = r3 + 43;// FIXED: was a reference to a temporary, now local
    int x2 = r5 + 44;// FIXED: was a reference to a temporary, now local
    int x3 = r6 + 45;// FIXED: was a reference to a temporary, now local
    return f1(f1(GLOBAL, variable 4), f1(local, constinit 2));// error: can't retu
    // NOTE: was temporary instead of local
}
```

This proposal and these three resolutions all satisfy the design group's opinion on safety for C++.

&✓	implicit constant	explicit constant	var	Opinion
✓	✓	✓	✓	<i>Do not radically break backwards compatibility – compatibility is a key feature and strength of C++ compared to more modern and more fashionable languages.</i> ^[14:10]
✓	✓	✓	✓	Do not deliver safety at the cost of an inability to express the abstractions that are currently at the core of C++ strengths. ^[14:11]
✓	✓	✓	✓	Do not leave us with a “safe” subset of C that eliminates C++’s productivity advantages. ^[14:12]
✓	✓	✓	✓	Do not deliver a purely run-time model that imposes overheads that eliminate C++’s strengths in the area of performance. ^[14:13]
✓	✓	✓	✓	Do not imply that there is exactly one form of “safety” that must be adopted by all. ^[14:14]
✓	✓	✓	✓	Do not promise to deliver complete guaranteed type-and-resource safety for all uses. ^[14:15]
✓	✓	✓	✓	Do offer paths to gradual and partial adoption – opening paths to improving the billions of lines of existing C++ code. ^[14:16]
✓	✓	✓	✓	Do not imply a freeze on further development of C++ in other directions. ^[14:17]
✓	✓	✓	✓	Do not imply that equivalent-looking code written in different environments will have different semantics (exception: some environments may give some code “undefined behavior” while others give it (a single!) defined behavior). ^[14:18]

What's more, `implicit constant` is the most effective. While both `&✓` and `implicit constant` can be categorized as `elimination`, only `implicit constant` automatically fixes instances of dangling in a non breaking way that is logical to end programmers.



While `explicit constant` and `var` are only `Personal Protective Equipment`, they are highly useful with or without this proposal as generic tools programmers can use to fix instances of dangling simply. They especially compliment this proposal as tools to fix dangling identified by this proposal.

Usage

A common complaint is that programmers would never use any attribution feature based on past experience with similar attribution efforts that existed outside of the standard. One usage of this proposal is applying its annotations to the STL itself. Taking just the `std::string` class as an example, following are the functions made safer with this proposal.

- Element access
 1. `at`
 2. `operator []`
 3. `front`
 4. `back`
 5. `data`

- 6. `c_str`
- 7. `operator basic_string_view`
- Iterators
 - 1. `begin`
 - 2. `cbegin`
 - 3. `end`
 - 4. `cend`
 - 5. `rbegin`
 - 6. `crbegin`
 - 7. `rend`
 - 8. `crend`
- Operations
 - 1. `insert`
 - 2. `insert_range`
 - 3. `erase`
 - 4. `append`
 - 5. `append_range`
 - 6. `operator +=`
 - 7. `replace`
 - 8. `replace_with_range`
- Input/output
 - 1. `getline`

The point is programmers who consume safer libraries don't have to do anything. The programmer of a library should apply this proposals annotations to its public top most API for documentation and if standardized for specification purposes. However, if a library producer does add it to its implementation than the library producer themselves would also benefit.

Viral Attribution Effort

Another common complaint with this and really any annotation based proposal is the work involved in annotating everything, in this proposal's case, functions. This attribution effort is just good documentation that a library producer should already be doing anyway to at least their public API. This proposal moves this documentation from non standard comments and/or

non standard external documentation to standardized annotations. Further, other static analysis tools produce mostly warnings which would require some type of viral attribution effort in the code to tell those tools to ignore false positives.

<pre>//NOSONAR</pre>	Sonar
<pre>@SuppressWarnings({"", "", ""})</pre>	Java ^[19] ^[20]
<pre>#pragma warning disable 0000 // code #pragma warning restore 0000</pre>	C# ^[21]
<pre>[SuppressMessage("", "", Justification = "")]</pre>	C# ^[21:1]

This proposal produces errors for definitely bad code and as such avoids the need for additional viral attribution efforts for ignoring false positives.

For this whole proposal, it has been advocated that library writers should provide such standardized documentation for the benefit of their consumers and the standardization effort. However, let’s entertain for the moment, if the library writer did not document their methods but instead their compilers did it for them i.e. **minimize attribution effort**. This would be performed when modules are compiled and the metadata/attributes retained in the module.

Direct reference to parameters buys programmers alternatives and default values.


```

const int& alternative(const int& left,
    const int& right)
{
    if(randomBool())
    {
        return left;
    }
    else
    {
        return right;
    }
}

const V& find_or_default(const std::map<K,V>& m,
    const K& key, const V& default_value)
{
    if(m.contains(key))
    {
        return m.get(key);
    }
    else
    {
        return default_value;
    }
}

```

```

[[dependson(left, right)]
const int& alternative(co
    const int& right)
{
    if(randomBool())
    {
        return left;
    }
    else
    {
        return right;
    }
}

[[dependson(default_value
const V& find_or_default(
    const K& key, const V
{
    if(m.contains(key))
    {
        return m.get(key)
    }
    else
    {
        return default_va
    }
}

```

By supporting `*this` programmers would obtain support for chained functions.

```

class S
{
public:
    S& assign()
    {
        return *this;
    }
    S& assign_alt(this S& self)
    {
        return self;
    }
};

```

```

class S
{
public:
    [[dependson(this)]]
    S& assign()
    {
        return *this;
    }
    [[dependson(this)]]
    S& assign_alt(this S& self)
    {
        return self;
    }
};

```

In the string class alone, operator=, assign, assign_range, insert, erase, append, operator+= and replace are already required by the standard to return *this.

By supporting member references programmers would obtain support for some getters.

```

class S
{
private:
    int i;
public:
    int& get()
    {
        return i;
    }
};

```

```

class S
{
private:
    int i;
public:
    [[dependson(this)]]
    int& get()
    {
        return i;
    }
};

```

By analyzing simple constructor member initializer lists, pointer/reference types can be detected.

```
class PointerReferenceIterator
{
private:
    int* p;
    int& r;
public:
    S(int* pp, int& rr) : p(pp), r(rr) {}
};
```

```
class PointerReferenceIterator
{
private:
    int* p;
    int& r;
public:
    [[dependson(pp, rr)]]
    S(int* pp, int& rr) : p(pp),
};
```

With this `string_view (s)`, `span (s)`, `function_ref (s)` can be recognized as being dependent upon some state.

All of the example expressions analyzed were simple instead of complex and combined provides significant less manual attribution and significant functional propagation of lifetime analysis.

Summary

The advantages of adopting said proposal are as follows:

1. Standardize the documentation of lifetimes of API(s) for developers
2. Standardize the specification of lifetimes for proposals
3. Produce more meaningful return error messages that doesn't involve value categories
4. Empowers programmers with tools to identify indirect occurrences of immediate dangling of references to the stack, simply
5. Empowers programmers with tools to identify indirect occurrences of return dangling of references to the stack, simply

Poll Results

R2

POLL: We should promise more committee time to pursuing this paper, knowing that our time is scarce and this will leave less time for other work

Favor	Neutral	Against
-------	---------	---------

—	—	—
5	12	18

Outcome: Consensus against

Frequently Asked Questions

Why not pointers?

References have to be initialized, can't be `nullptr` and can't be rebound which means by default the lifetime of the instance the reference points to is fixed at the moment of construction which has to exist lower on the stack i.e. prior to reference creation which is known at compile time. This is very safe by default. Pointers and reference classes that has pointer semantics are none of these things. Since they are so dynamic, the relevant metadata would more frequently be needed at run time.

That having been said, pointers used in a reference like way could benefit from the checks proposed in this paper.

“a reference is similar to a const pointer such as `int* const p` (as opposed to a pointer to const such as `const int* p`)” ^[22]

How can you reseal a reference to make it refer to a different object? ^[22:1]

ISOCPP > Wiki Home > References

This would require increasing the two bits of metadata to include a third bit in order to handle `nullptr` and uninitialized. This is not currently in scope of this proposal.

Why not **explicit lifetime dependence on struct (s)?**

1. References are rarely used in class definitions. Instead programmers use `std::reference_wrapper`, smart pointers and plain old pointers. Since all of these are rebindable, they are more runtime than compile time and as such is not the subject of this proposal. See [why not pointers?](#) for more information.
2. Explicit lifetime dependence on `struct (s)/ class [es]` breaks abstraction because the user of a library need to know class/struct implementation details that are likely `protected`, `private` and `internal`. For public reference members of `struct` instances, the compiler with these enhancements can already propagate these references. Further,

explicit lifetime dependence on functions allows propagating the needed information when applied to constructors, conversion operators and factory functions.

References

1. <https://stackoverflow.com/questions/31609137/why-are-explicit-lifetimes-needed-in-rust> (<https://stackoverflow.com/questions/31609137/why-are-explicit-lifetimes-needed-in-rust>) ↔ ↔
2. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf>) ↔
3. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2742r2.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2742r2.html>) ↔
4. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2771r0.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2771r0.html>) ↔
5. <https://www.cdc.gov/niosh/topics/hierarchy/default.html> (<https://www.cdc.gov/niosh/topics/hierarchy/default.html>) ↔
6. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#inforce-enforcement> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#inforce-enforcement>) ↔ ↔ ↔
7. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p5-prefer-compile-time-checking-to-run-time-checking> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#p5-prefer-compile-time-checking-to-run-time-checking>) ↔
8. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1179r1.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1179r1.pdf>) ↔ ↔ ↔
9. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2806r1.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2806r1.html>) ↔ ↔

10. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2680r1.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2680r1.pdf>) ↔ ↔ ↔
11. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html>) ↔ ↔
12. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf>) ↔ ↔
13. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0792r14.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0792r14.html>) ↔
14. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2759r1.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2759r1.pdf>) ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔ ↔
15. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2724r1.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2724r1.html>) ↔
16. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r1.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r1.html>) ↔
17. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3038.htm> (<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3038.htm>) ↔
18. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r1.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r1.html>) ↔
19. <https://docs.oracle.com/javase/8/docs/api/java/lang/SuppressWarnings.html> (<https://docs.oracle.com/javase/8/docs/api/java/lang/SuppressWarnings.html>) ↔
20. <https://www.baeldung.com/java-suppresswarnings> (<https://www.baeldung.com/java-suppresswarnings>) ↔

21. <https://stackoverflow.com/questions/1378634/is-there-a-way-to-suppress-warnings-in-c-sharp-similar-to-javas-suppresswarnin> (<https://stackoverflow.com/questions/1378634/is-there-a-way-to-suppress-warnings-in-c-sharp-similar-to-javas-suppresswarnin>) ↔ ↔

22. <https://isocpp.org/wiki/faq/references#reseating-refs>
(<https://isocpp.org/wiki/faq/references#reseating-refs>) ↔ ↔